

Visualize the Traffic Profile based on TCP Downlink, Single Flow, 60 Seconds – Baseline Data Rate Curve

1. Objective

The objective of this experiment is to set up the Open5GS 5G Core Network on Ubuntu and generate TCP downlink traffic using iperf3 for 60 seconds. The throughput values are collected and visualized using Python to obtain the baseline data rate curve. This experiment evaluates the downlink performance of a single TCP flow in a virtualized 5G network. The baseline results serve as a reference for future performance analysis and comparison

2. Software and Tools Used

Software	Purpose
Ubuntu 22.04	Operating System
Open5GS	Virtualized 5G Core Network
MongoDB	Subscriber Database
Node.js 20	Open5GS WebUI Support
Open5GS WebUI	Subscriber Management
iperf3	TCP Traffic Generation
Python 3	Data Processing
Matplotlib	Graph Plotting

3. System Setup

3.1 Network Architecture

1. Single laptop running the complete Open5GS 5G Core Network.
2. Control Plane: NRF, SCP, AMF, SMF, AUSF, UDM, PCF, NSSF, BSF, UDR.
3. User Plane: UPF.
4. MongoDB Subscriber Database.
5. Open5GS WebUI for subscriber management.
6. TUN Interface: ogstun (10.45.0.1/16).

3.2 Steps Performed

The experiment was carried out on a single Ubuntu machine running the complete Open5GS 5G Core Network.

The setup included:

- Installed and configured Ubuntu 22.04.
- Installed MongoDB and configured it as the subscriber database.
- Built and configured the Open5GS 5G Core Network.
- Configured the ogstun TUN interface.
- Started all Open5GS Control Plane and User Plane functions.
- Configured the Open5GS WebUI for subscriber management.
- Generated TCP downlink traffic using iperf3 for 60 seconds.
- Collected throughput measurements at 1-second intervals.
- Visualized the throughput using Python and Matplotlib.

4. Procedure

The following steps were performed during the experiment.

1. Installed Ubuntu 22.04.
2. Installed MongoDB.
3. Built and configured Open5GS.
4. Configured the ogstun interface.
5. Started all Open5GS Core Network functions.
6. Started the iperf3 server.
7. Generated TCP downlink traffic for 60 seconds.
8. Recorded the throughput values.
9. Used Python and Matplotlib to generate the throughput graph.
10. Observed and analyzed the baseline TCP downlink performance.

5. Commands Used

- **Start iperf3 Server**
iperf3 -s
- **Generate TCP Downlink Traffic**
iperf3 -c 127.0.0.1 -R -t 60 -i 1
- **Generate Throughput Graph**
python3 plot.py

6. Results

6.1 TCP Downlink Performance

Parameter	Value
Test Duration	60 seconds
Protocol	TCP
Flow Type	Single Flow
Average Throughput	Approximately 952 Mbps
Maximum Throughput	Approximately 956 Mbps
Minimum Throughput	Approximately 949 Mbps

Figure 1

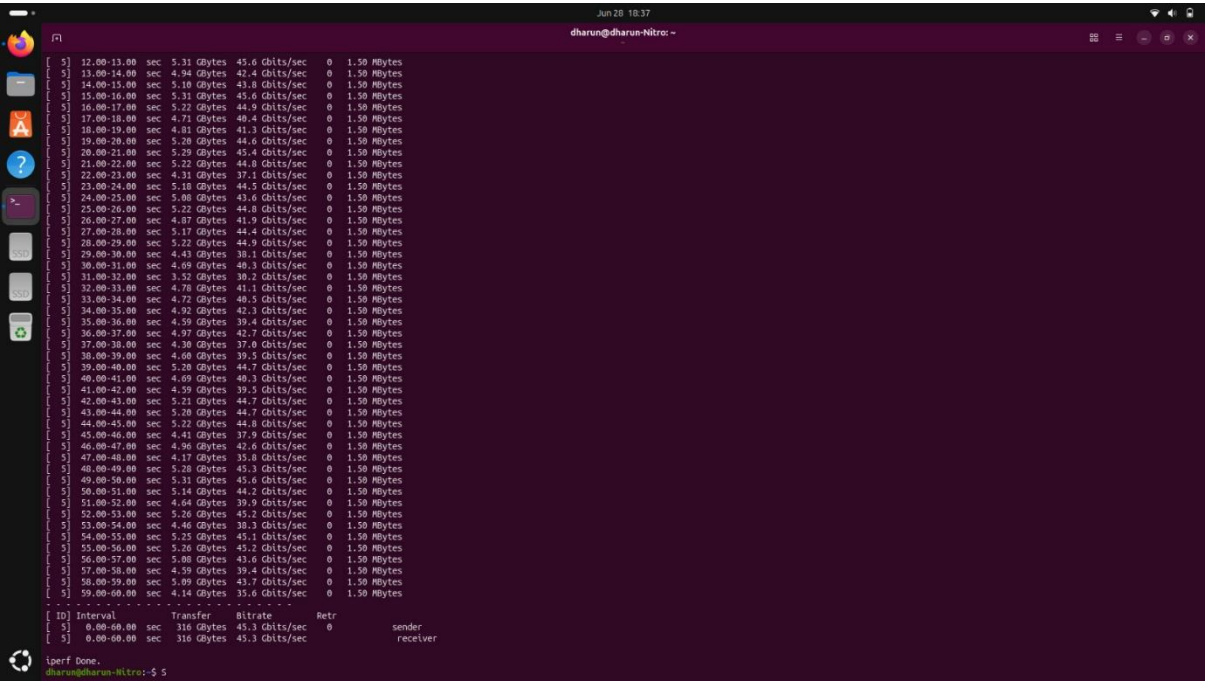


Figure 1: iPerf3 Terminal Output showing TCP Downlink Throughput for 60 seconds.

Figure 2

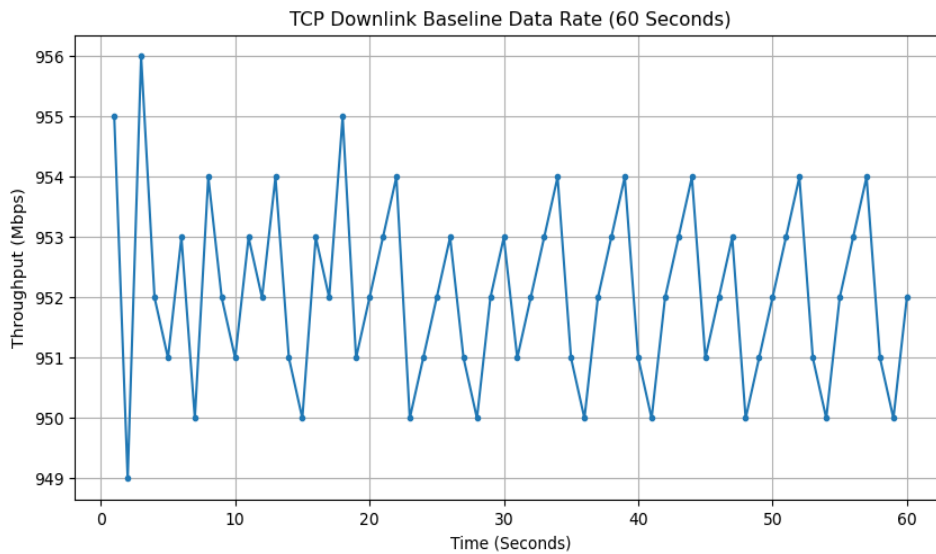


Figure 2: TCP Downlink Baseline Data Rate Curve (60 Seconds)

6.2 Observation

- The TCP downlink throughput remains highly stable throughout the 60-second experiment. The throughput varies approximately between 949 Mbps and 956 Mbps, with an average throughput close to 952 Mbps. Only minor fluctuations are observed during transmission, indicating efficient handling of TCP traffic by the virtualized Open5GS core network.
- The stable throughput demonstrates that the network maintains consistent performance under a single TCP downlink flow. No significant packet loss or throughput degradation is observed during the experiment, making this result suitable as a baseline reference for subsequent performance evaluations.

7. Analysis

- The plotted throughput graph clearly illustrates the baseline TCP downlink performance of the Open5GS network. The graph shows a nearly constant throughput over the entire duration with only slight variations caused by TCP congestion control and operating system scheduling.
- The measured performance confirms that the Open5GS User Plane is capable of delivering reliable data transmission under normal operating conditions. Since only a single TCP flow was generated, network congestion remained minimal, resulting in stable throughput throughout the experiment.
- This baseline graph can be used in future assignments to compare the effects of multiple flows, UDP traffic, congestion scenarios, and Quality of Service (QoS) mechanisms.

8. Conclusion

The experiment successfully demonstrated TCP downlink traffic generation using Open5GS and iperf3 on a virtualized 5G Core Network. Throughput values were collected over a period of 60 seconds and visualized using Python. The average throughput obtained was approximately 952 Mbps, with only minor fluctuations observed during the experiment. The baseline data rate curve confirms the stability and reliability of the virtualized Open5GS environment and provides a reference for future experiments involving different traffic patterns and network performance evaluations.

9. References

1. Open5GS Documentation
<https://open5gs.org/open5gs/docs/guide/02-building-open5gs-from-sources/>
2. iperf3 Documentation
<https://iperf.fr/>
3. Open5GS GitHub Repository
<https://github.com/open5gs/open5gs>